Is Narration Studio harness-engineered?

Whether this repository is “harness engineered,” measured against the actual code, and what to do next so a later update can fully adopt harness engineering without a rewrite.

> **Confirmed** means the behavior is in this repository’s source. **Inferred** means it comes from Microsoft Agent Framework (MAF) or from the industry definition of an agent harness.

Verdict

Partially. Narration Studio has an agent harness for *producing videos*. It is not harness-engineered as a *coding-agent* repository.

There is no `AGENTS.md`, Cursor rules, skills, test project, or CI that an IDE agent could use to check its own work. The word “harness” never appears in the source.

What *is* here is a production harness around the model: the LLM does not own the work. The loop, tools, context, and controls do.

What “harness” means here

An agent harness is the scaffold around a model: the loop that keeps calling it, the tools it may invoke, the context injected into prompts, and the controls that stop unsafe or incomplete work. Claude Code, Cursor, and Codex are harnesses in that sense. Microsoft Agent Framework is a **Harness-as-a-Service** library (**inferred**): this repo customizes it rather than writing its own chat-tool loop.

A common inclusion test (**inferred**, not a type in this repo) is four pieces:

Piece	Question
Loop	Does something keep the model turning until a stop condition?
Tools	Can the model only act through a defined interface?
Context	Is live task state injected, not just a static system prompt?
Control	Can the run be limited, filtered, or failed over without trusting the model?

Production harness (present)

Loop

`WorkflowHost` builds MAF **handoff** or **sequential** graphs. Handoff uses a coordinator with autonomous mode, `turnLimit: 8`, and a continuation prompt that says to call `GetPipelineStatus` and stop when the merged video exists. Sequential runs specialists in a fixed order.

Confirmed: if agents finish without a merged file, `ProductionApp` falls through to `DirectPipelineRunner`. Direct mode is itself a loop: a fixed list of tool calls, no LLM routing.

Default CLI mode is `handoff`. Passing `--orchestration direct` skips MAF entirely.

Tools

`ProductionTools` is the real pipeline. Agents receive `AIFunction` wrappers of the same methods that direct mode calls. Specialists do not write PNGs, MP3s, or MP4s themselves; they call tools that do.

That is the load-bearing harness pattern in this repo: **tools are the product; agents are optional drivers.**

Context

Every specialist in `AgentFactory.TryCreate` gets two `AIContextProvider`s:

1. `ProductionContextProvider` — injects `PipelineStore.SummarizeForAgent()` plus naming/cap instructions before a turn; appends truncated assistant text to `PipelineState.Notes` afterward (audit log, not retrieval).
2. MAF `FileMemoryProvider` — store under `output/agent-file-memory/production-notes-{RunId}`. Constructed only in `TryCreate`, so **direct mode never starts it**.

Independently, `PipelineStore` writes `run.txt`, `pipeline-status.txt`, and `notes.txt` into that timestamped folder on every persist, including direct mode. Older `production-notes-*` folders are kept. See `docs/memory-vs-mem0.md`.

Control

Control	Where	What it actually does
Named mutex `NarrationStudio.Production`	`ProductionApp`	One run at a time
Guardrail middleware	`ProductionGuardrails` on each MAF agent	Blocked-term short-circuit; email/phone redaction on chat messages; tool start/finish logs
Prompt “Guardrails you must obey”	`ProductionContextProvider`	Advisory text only
Hard 8-script / 5-slide caps	`ProductionTools` throws / `Take()`	Enforced whether or not agents run
Failover to direct	`ProductionApp`	Completes remaining tools if merge is missing
Wipe media on start	`PipelineStore.Initialize`	Prevents stitching old audio/video onto a new talk

Guardrails wrap agents only. Direct mode does not run `ProductionGuardrails`. The slide-count “trim” log does **not** change the tool result. Fuller write-up: `docs/guardrails.md`.

Coding-agent outer harness (absent)

Harness engineering for *Cursor / Codex / Claude Code* usually means repo-owned assets the coding agent can follow and verify: `AGENTS.md`, rules, skills, linters, tests, CI.

This repository has none of those. A coding agent working on Narration Studio is steered by chat and by reading the C# — not by a checked-in outer harness.

What is missing if you mean a complete harness

- No evals or tests the agent (or a developer) can run to self-verify a run.
- Guardrails do not apply in direct mode; slide-count middleware only logs.
- Slide-sentence Ollama and TTS pass `CancellationToken.None`; Host stop is wired into `WorkflowHost` only.
- Memory is a status snapshot plus files, not fact extraction or similarity search.
- Several agent prompts still hardcode "8" even if `Studio:SegmentCount` changes.
- `UseStubNarration` skips the 7-minute Ollama rewrite but **not** per-slide Ollama sentences.

How the two modes use the harness

```
handoff / sequential
    → AgentFactory.TryCreate
        → tools + context providers + FileMemoryProvider + guardrail middleware
    → WorkflowHost loop
    → if no merged MP4 → DirectPipelineRunner (same tools, no MAF)

direct (or TryCreate fails)
    → DirectPipelineRunner
        → ProductionTools in order
        → PipelineStore snapshots still written
        → no FileMemoryProvider, no ProductionGuardrails
```

The durable contract is `ProductionTools` plus `DirectPipelineRunner`, not the LLM graph.

What should be done to fully embrace harness engineering

This app can be prepared **without a rewrite**. The durable piece is already the right one: `ProductionTools` owns I/O; agents only drive it. Full harness engineering here means tightening that contract so a later model, workflow, or IDE agent can change without the factory changing.

Treat two layers separately. Mixing them is how harness work turns into a mess.

Production harness (the video factory)

This is what "fully embraced" should mean for Narration Studio.

Already in place: tool interface, handoff/sequential loop, failover to `DirectPipelineRunner`, pipeline-state context, mutex, wipe-and-regenerate media.

Still needed:

Gap	Why it matters	Prepare now
Caps live in prompts as "8" / `audio-01..08`	Changing `SegmentCount` lies to the model	Generate agent instructions and tool `[Description]`s from `StudioOptions`
Direct mode bypasses guardrails	Two control planes	Put policy in tools (already true for 8/5). Keep middleware as chat-only, or run the same validator on tool entry in every mode
Slide "trim" only logs	Control that does not control	Either enforce (truncate/reject) or delete the warning
`CancellationToken.None` on Ollama/TTS	Host stop cannot halt the expensive stages	Thread `ApplicationStopping` through `ProductionTools` now, even if behavior is otherwise unchanged
No machine check that a run is "done"	Harness without a sensor	Add one eval: given a fixture transcript, assert 8 scripts, ≤5 slides, 8 mp3s, merged mp4 exists, `pipeline-state.json` status `complete`
No traces	You cannot improve the harness from failures	Write a JSONL event log per run (tool name, duration, success/fail) next to `production-notes-{RunId}`
Slide Ollama has no flag	Stubbing looks like "no LLM" but slides still need Ollama	Add `UseStubSlidePhrases` (or similar) so LLM stages are explicit and skippable
Memory is snapshot-only	Fine for a factory; do not pretend it is mem0	Keep snapshots. If you later want retrieval, add it *beside* `PipelineStore`, not inside it

The stop condition should be **files on disk**, not the coordinator's last sentence. That is already how failover works; make it the official definition of success.

Coding-agent outer harness (Cursor / Codex working on this repo)

This is optional unless humans will keep changing the app with agents.

Prepare with cheap, portable files:

- `AGENTS.md` — working directory is repo root, `dotnet run --project src\NarrationStudio -- --orchestration direct`, do not commit `appsettings` secrets, tools not prompts own the pipeline
- A few Cursor rules that point at `ProductionTools` / `PipelineStore` / "do not hardcode 8"
- A `dotnet test` project with the eval above so an agent can self-verify
- No CI is required to *prepare*; a test the agent can run is

Do not wait on skills/MCP. Those are later multipliers.

Can it be prepared without updating the product yet?

Yes. The preparation is structural, not a new video feature.

Do now (low risk, high leverage):

1. Freeze the tool contract. `ProductionTools` remains the only way to touch disk/APIs. New stages (music, captions) are new methods, not agent-side file writes.
2. Drive all "8 / 5 / 7 minutes" from options. Prompts, handoff text, and file-name examples interpolate `SegmentCount` / `MaxSlidesPerSegment`.
3. Pass a cancellation token through rewrite, slide sentences, TTS, and ffmpeg.
4. Define success as artifacts. One `HarnessChecks` (or test) that inspects `output/` after a fixture run.
5. Structured run events in the timestamped memory folder (that folder is already created every run).
6. `AGENTS.md` plus the existing companion PDFs so a coding agent inherits the same mental model.

Defer until you actually update:

- Real eval suite / golden PNG hashes
- mem0-style retrieval
- Guardrails on direct mode (only if untrusted input will hit that path)
- Parallel TTS/slides
- Harness-as-a-Service swap (keep MAF; do not rewrite the loop until the tool+eval layer is stable)

What "fully embraced" looks like later

```
fixture transcript
    → DirectPipelineRunner or handoff
    → ProductionTools only
    → events.jsonl + pipeline-state.json + production-notes-{RunId}
    → HarnessChecks: 8/5/merged/status
    → fail the process if checks fail (even if the model said "Done")
```

Agents become interchangeable. Direct mode stays the reference implementation. That is the whole point of harness engineering for this app.

Do not invert the current shape (do not let specialists write files). The scaffold a later update would hang on is: `AGENTS.md` + option-driven prompts + a single post-run check.

The option of going further — critic, resume, human approve, traces, recovery — is a **product** change, not only a harness change. See `docs/agentic-system.md`.

Key files

File	Role
`src/NarrationStudio/ProductionApp.cs`	Mode selection, mutex, stub-before-agents, failover to direct
`src/NarrationStudio/Orchestration/DirectPipelineRunner.cs`	Deterministic tool loop
`src/NarrationStudio/Tools/ProductionTools.cs`	Shared tool interface; hard caps
`src/NarrationStudio/Agents/AgentFactory.cs`	Crew, tools, context, file memory, guardrails
`src/NarrationStudio/Agents/WorkflowHost.cs`	Handoff / sequential loop
`src/NarrationStudio/Memory/ProductionContextProvider.cs`	Injected status + prompt policy
`src/NarrationStudio/Guardrails/ProductionGuardrails.cs`	Agent-only message filter
`src/NarrationStudio/Services/PipelineStore.cs`	Run state, wipe policy, memory snapshots